

ANDROID STUDIO

KOTLIN-ZERRENDATZE-MOTAK

HELBURUAK

Gai honetan kontzeptu hauek landuko ditugu:

1. Balioen tartea mugatzea
2. Zerrendatze-mota bat sortzea
3. Zerrendatze-mota baten erabilera
4. Zerrendatze-moten erabilera "when" baldintzetan
5. Propietateak eta funtzioak eranstea zerrendatze-motei
6. Propietate integratuak

BALIOAK MUGATUZ

Atal honetan, klase mota berezi bat ikusiko dugu (zerrendatze-mota bat), bereziki erabilgarria dena balio kopuru mugatu bat adierazi nahi denean.

String aldagai bat sortuko dugu Schnauzer maskota baten izena gordetzeko:

```
val nameOfSchnauzer: String = "Shadow"
```

Zenbat izen ezberdin bururatzen zaizkizu Schnauzertarrentzat? Niri Shadow, Rover eta Captain Fluffybeard datozkit burura, baina egia esan ez dago mugarik eman dakizkiekeen izenen barietatearentzat. Aukerak mugagabeak dira!

BALIOAK MUGATUZ

Schnauzerren arrazak, ordea, istorio ezberdina dira. 3 Schnauzer arraza bakarrik daude:

- ▶ Schnauzer miniatura
- ▶ Schnauzer estandarra
- ▶ Schnauzer erraldoia

Schnauzer batentzako izen kopurua mugagabea den arren, arraza kopurua 3 aukeratarara mugatuta dago.

BALIOAK MUGATUZ

Nola lotzen da hori guztia programazioarekin?

Kodea idazten dugunean, errorearen probabilitatea murriztu dezakegu, balio posibleen tartea balio baliodunetara soilik mugatzen duen mota bat hautatuz. Adibidez, String batek ia imajina dezakezun edozein testu eduki dezake. Zure txakurrari ia edozein izen jar diezaiokezunez, zentzuzkoa litzateke String bat erabiltzea bere izena jartzeko. Bestalde, String bat ez zen aukera ona izango Schnauzer baten arraza espezifikoki eusteko. Zergatik? Kateek balio kopuru infinitua izan dezaketelako, baina 3 aukera baino ez daude Schnauzer arraza batentzat zuzenak izan daitezkeenak. Beste era batera esanda, ezin da bermatu hiru balio zuzen horietako batean arraza-aldagai bat ezartzea. Ikus dezagun adibide bat hori frogatzeko.

BALIOAK MUGATUZ

Demagun arraza definitzeko String bat erabiltzea erabakitzen dugula, honela:

```
val breedOfSchnauzer: String = "Mini"
```

Orain, gure kodeko beste leku batean, arraza honela egiaztatzen duen baldintza bat izan genezake:

```
if (breedOfSchnauzer == "Miniature") {  
    // ... do something  
}
```

BALIOAK MUGATUZ

Baldintzapeko adierazpena true izateko asmoa izan genezakeen, baina "Mini" idatzi beharrean "Miniature" idatzi dugunez, faltsutzat hartuko zen... eta ez genuen jakingo zerbait gaizki zegoela kodea exekutatu arte.

Arazoa da balio mugagabeak ahalbidetzen dituen mota bat erabiltzen dugula (hau da, String mota), soilik balio kopuru mugatua ahalbidetzen duen mota bat behar genuenean. Hiru arraza-izenetako bat bakarrik erabiltzeko aukera emango dizun tipo bat sortu ahal izango bagenu, aurreko akatsa posible ere ez izatea lortuko genuke!

Kotlinek balio mugatuak dituen mota bat sortzeko, **zerrendatze-mota** bat erabiltzen da (**enumeración**)

ZERRENDATZE-MOTA BAT SORTZEA

Zerrendatze klase bat sortuko dugu Schnauzer arrazak definitzeko.

```
enum class SchnauzerBreed {  
    MINIATURE,  
    STANDARD,  
    GIANT  
}
```

Honek dira definizioaren atal garrantzitsuenak:

The diagram shows the code from the previous block with annotations. A bracket labeled 'keywords' points to 'enum class'. A bracket labeled 'class name' points to 'SchnauzerBreed'. A bracket labeled 'enum entries' points to the three entries: 'MINIATURE', 'STANDARD', and 'GIANT'.

```
enum class SchnauzerBreed {  
    MINIATURE,  
    STANDARD,  
    GIANT  
}
```


ZERRENDATZE-MOTA BAT SORTZEA

1. Zerrendatze-mota bat sortzeko, **enum class** idatziko dugu, eta ez soilik class.
2. Honen ondoren, gure kalseari eman nahi diogun izena dator, kasu honetan, **SchnauzerBreed**.
3. Klasearen gorputzaren barruan dauden aukerei **zerrendatze-sarrerak** esaten zaie. Batzuetan, zenbaketa-konstanteak esaten diegu. Komekin bereizten dira.

ZERRENDATZE-MOTA BAEN ERABILERA

Kontuan izan behar dugu Zerrendatze batetik abiatuta ez dugula objektu bat eraikiko klase normalekin egingo genukeen moduan, hórrela egin eskero errorea emango luke:

```
val breed: SchnauzerBreed = SchnauzerBreed()
```

Horren ordeez, aldagai bati esleitzen dio aukeretako bat, honela:

```
val breed: SchnauzerBreed = SchnauzerBreed.GIANT
```

ZERRENDATZE-MOTA BAEEN ERABILERA

Kontuan izan behar dugu Zerrendatze batetik abiatuta ez dugula objektu bat eraikiko klase normalekin egingo genukeen moduan, hórrela egin eskero errorea emango luke:

```
val breed: SchnauzerBreed = SchnauzerBreed()
```

Horren ordeez, aldagai bati esleitzen dio aukeretako bat, honela:

```
val breed: SchnauzerBreed = SchnauzerBreed.GIANT
```

ZERRENDATZE-MOTA BAEEN ERABILERA

Beraz zerrendatze-mota bat erabiltzean, MINIATUREren ordeez MINI idatziko bagenu, Kotlin-ek errore bat emango liguke kodea exekutatu aurretik.

```
val breed: SchnauzerBreed = SchnauzerBreed.MINI
```

ZERRENDATZE-MOTEN ERABILERAKO

"WHEN" BALDINTZETAN

Zerrendatze-klaseak erabiltzearen beste onura interesgarri bat da Kotlin-ek laguntza gehigarri bat ematen duela *when* baldintza batekin erabiltzen ditugunean.

Gogoratuko dugunez, adierazpenek baldintza guztiak hartu behar dituzte kontuan, hau da, zehatzak izan behar dute. Zerrendatze-motek balio posibleak mugatzen dituztenez, Kotlin-ek informazio hori erabil dezake baldintza guztiak noiz hartu ditugun kontuan jakiteko.

Adibidez, hemen arraza baten deskribapena itzultzen duen kode bat dago.

ZERRENDATZE-MOTEN ERABILERAK

"WHEN" BALDINTZETAN

```
fun describe(breed: SchnauzerBreed) = when (breed) {  
    SchnauzerBreed.MINIATURE -> "Small"  
    SchnauzerBreed.STANDARD  -> "Medium"  
    SchnauzerBreed.GIANT     -> "Large"  
}
```

Kontuan izan hemen ez dagoela else-rik! Kotlin-ek jakin dezake hiru zerrenda-sarrerak sartu ditugula when gorputzean, eta, beraz, ez dago beste aukerarik. Sarrera batzuk alde batera utziko bagenitu, Kotlin-ek errore bat emango liguke:

```
fun describe(breed: SchnauzerBreed) = when (breed) {  
    SchnauzerBreed.MINIATURE -> "Small"  
    SchnauzerBreed.STANDARD  -> "Medium"  
}
```

ZERRENDATZE-MOTEN ERABILERAKO "WHEN" BALDINTZETAN

Errorea konpondu ahal izateko:

```
fun describe(breed: SchnauzerBreed) = when (breed) {  
    SchnauzerBreed.MINIATURE -> "Small"  
    SchnauzerBreed.STANDARD  -> "Medium"  
    else                       -> "Unknown"  
}
```

PROPIETATEAK ETA FUNTZIOAK ERANSTEA ZERRENDATZE-MOTEI

Posible al da propietateak eta funtzioak zerrendatze-mota bati eranstea? **BAI**

```
enum class SchnauzerBreed(val height: Int) {  
    MINIATURE(33),  
    STANDARD(47),  
    GIANT(65)  
}
```

Height izeneko eraikitzaile-parametro berri bat gehitu dugunez, eraikitzaile-argumentu bat gehitu behar diogu zerrenda-sarrera bakoitzari.

constructor parameter

↓

```
enum class SchnauzerBreed(val height: Int) {  
    MINIATURE(33),  
    STANDARD(47),  
    GIANT(65)  
}
```

constructor arguments

The diagram illustrates the relationship between the constructor parameter and the constructor arguments. An arrow points from the text 'constructor parameter' to the 'val height: Int' part of the constructor. Another arrow points from the text 'constructor arguments' to the values 33, 47, and 65 in the constructor arguments of the enum entries.

PROPIETATEAK ETA FUNTZIOAK ERANSTEA ZERRENDATZE-MOTEI

Zerrendatze-instantzia baten propietate bat lortzeko, objektu normalekin egingo genukeen bezala da. Adibidez, arraza baten altuera inprima dezakegu pantailan, honela:

```
println(SchnauzerBreed.MINIATURE.height)
```

Eraikitzailearen argumenturik behar ez duen propietate bat ere sar dezakegu. Erantsi dezagun jabetza bat, guztiak zein family arrazatakoak diren adieraziko diguna.

```
enum class SchnauzerBreed(val height: Int) {  
    MINIATURE(33),  
    STANDARD(47),  
    GIANT(65);  
  
    val family: String = "Schnauzer"  
}
```

PROPIETATEAK ETA FUNTZIOAK ERANSTEA ZERRENDATZE-MOTEI

Family propietate berria klasearen gorputzaren amaieran gehitzen da. Garrantzitsua da kontuan izatea azken zerrendatze-sarreraren ondoren puntu eta koma behar dela. Kotlin-en ez da askotan puntu eta koma bat erabili behar izaten, baina hau horietako bat da.

```
enum class SchnauzerBreed(val height: Int) {  
    MINIATURE(33),  
    STANDARD(47),  
    GIANT(65);  
  
    val family: String = "Schnauzer"  
}
```

PROPIETATEAK ETA FUNTZIOAK ERANSTEAK ZERRENDATZE-MOTEI

Funtzio bat gehitzea ere erraza da.

```
enum class SchnauzerBreed(val height: Int) {  
    MINIATURE(33),  
    STANDARD(47),  
    GIANT(65);  
  
    val family: String = "Schnauzer"  
  
    fun isShorterThan(centimeters: Int) = height < centimeters  
}
```

PROPIETATEAK ETA FUNTZIOAK ERANSTEA ZERRENDATZE-MOTEI

Propietateak lor ditzazkegu edo funtzioei deitu, beste edozein motarekin egingo genukeen bezala.

```
println(SchnauzerBreed.STANDARD.family)           // Prints "Schnauzer"  
println(SchnauzerBreed.STANDARD.isShorterThan(40)) // Prints "false"
```

PROPIETATE INTEGRATUAK

Guk definitutako edozein propietatez gain, Kotlin-ek ere automatikoki gehitzen ditu pare bat propietate zerrendatze-instantzia guztietan.

Ordinal: Zerrendan zerrenda-sarrera hori non agertzen den adierazteko erabil dezakegu.

```
enum class SchnauzerBreed(val height: Int) {  
    0 → MINIATURE(33),  
    1 → STANDARD(47),  
    2 → GIANT(65)  
}
```

Name: Zerrenda-sarreraren izena jakin behar baduzu, kate bat bezala.

```
fun describe(breed: SchnauzerBreed) {  
    println(breed.name)  
    println(breed.height)  
}
```

```
ESTÁNDAR  
47
```

PROPIETATE INTEGRATUAK

Guk definitutako edozein propietatez gain, Kotlin-ek ere automatikoki gehitzen ditu pare bat propietate zerrendatze-instantzia guztietan.

Ordinal: Zerrendan zerrenda-sarrera hori non agertzen den adierazteko erabil dezakegu.

```
enum class SchnauzerBreed(val height: Int) {  
    0 → MINIATURE(33),  
    1 → STANDARD(47),  
    2 → GIANT(65)  
}
```

Name: Zerrenda-sarreraren izena jakin behar baduzu, kate bat bezala.

```
fun describe(breed: SchnauzerBreed) {  
    println(breed.name)  
    println(breed.height)  
}
```

```
ESTÁNDAR  
47
```